



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 - HIGH PERFORMANCE DATA PROCESSING

SECTION 2

PROJECT 1:

**OPTIMIZING HIGH-PERFORMANCE DATA PROCESSING FOR LARGE SCALE
WEB CRAWLERS**

LECTURER: DR. SEAH CHOON SEN

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160

TABLE OF CONTENTS

1.0 INTRODUCTION	4
1.1 Background of the Project	4
1.2 Objectives	4
1.3 Target Website and Data to be Extracted	4
2.0 SYSTEM DESIGN AND ARCHITECTURE	5
2.1 System Architecture	5
2.2 Tools and Frameworks	6
2.3 Team Responsibilities	7
3.0 DATA COLLECTION	7
3.1 Crawling Methodology	7
3.2 Dataset Volume and Progressive Storage	9
3.3 Ethical Considerations and Server Etiquette	10
4.0 DATA PROCESSING	11
4.1 Overview	11
4.2 Cleaning Steps Applied	11
4.3 Records Before and After Cleaning	12
4.4 Final Dataset Structure	13
4.5 Derived Columns and Purpose	13
5.0 OPTIMIZATION TECHNIQUES	14
5.1 Compute & Memory Data Processing Optimization	14
5.2 Implementation Summary	15
5.3 Optimization Verification	15
6.0 PERFORMANCE EVALUATION	15
6.1 Baseline Performance (Data Cleaning Stage)	15

6.2 Optimized Performance (Data Cleaning Stage - Multiprocessing)	16
7.0 CHALLENGES AND LIMITATIONS	19
7.1 Challenges	19
7.1.1 Issues faced in Data Crawling	19
7.1.2 Issues faced in Data Cleaning	19
7.1.3 Issues faced in Multiprocessing Optimization	20
7.2 Limitations	20
7.2.1 Limitations in Data Crawling	21
7.2.2 Limitations in Data Cleaning	21
7.2.3 Limitations in Multiprocessing Optimization	21
8.0 CONCLUSION AND FUTURE WORK	21
APPENDICES	23

1.0 INTRODUCTION

This section introduces the project by outlining the background, objectives, and the data to be extracted. It provides an overview of the development of a large-scale web crawling and data processing pipeline for collecting news articles from the BERNAMA website, followed by data cleaning, optimization, and performance evaluation.

1.1 Background of the Project

The project was developed as a part of the High Performance Data Processing (HPDP) course. The goal of the course is to provide students hands-on experience in the design and optimization of large-scale data processing systems. For the project, the students need to build a complete data pipeline to collect, process, and store at least 100,000 structured records from an approved Malaysian website and evaluate its performance using high-performance computing techniques.

For this project, the BERNAMA English news website was selected as the data source. A baseline web crawling pipeline was developed to collect news articles, followed by a data cleaning process to improve data quality and consistency. The pipeline was further optimized using parallel processing techniques, which is multiprocessing during data processing. The optimized data processing is then evaluated against the baseline processing based on performance metrics such as execution time, throughput, CPU utilization, and memory usage.

1.2 Objectives

The objectives of this project are:

1. To develop an automated web crawler that collects large-scale news articles from the BERNAMA website.
2. To extract structured information including the headline, publication date, section, article summary, and URL from each news article.
3. To perform data cleaning by removing duplicate records, standardizing data formats, validating extracted data, and producing a high-quality dataset.
4. To optimize the data processing pipeline using high-performance computing techniques,

specifically multiprocessing, to improve processing efficiency.

5. To evaluate the performance differences between the baseline and optimized processing using execution time, throughput, CPU utilization, and memory usage measurements.

1.3 Target Website and Data to be Extracted

The target website selected for this project is the English news portal of BERNAMA, Malaysia's national news agency. The website publishes a wide range of news articles covering local and international events across various topics. Its structured webpage layout and extensive collection of archived articles make it suitable for large-scale web crawling, allowing the collection of more than 100,000 news records required for the project. The crawler accesses individual news articles using sequential article identifiers embedded in the URL, allowing efficient extraction without repeatedly traversing category pages.

To produce a structured dataset, the crawler extracts the following information in Table 1.3.1 from each article:

Table 1.3.1: Extracted Data Information

Data Field	Description
Headline	The title of the news article.
Publication Date	The date and time the article was published.
Section	The category or section of the news article.
Article Summary	The first meaningful paragraph representing a brief summary of the article.
URL	The unique web address of the news article.

The extracted data is stored in a CSV file before undergoing data cleaning and preprocessing. The cleaning process removes duplicate and invalid records and standardizes data formats and later was optimized using parallel processing techniques, and comparisons between baseline and optimized processing techniques are done using suitable visualization for performance analysis.

2.0 SYSTEM DESIGN AND ARCHITECTURE

This section outlines the overview of the system architecture, including the overall architecture, the tools and frameworks used, and also the team responsibilities for the project.

2.1 System Architecture

The proposed system consists of three main stages: data collection, data processing, and performance optimization and evaluation.

- 1. **The Data Collection Stage:** This stage involves developing a web crawling module to collect news articles from the BERNAMA English news portal. The crawler generates article URLs using sequential article identifiers, retrieves webpage contents, and extracts the required article information into a structured raw dataset.
- 2. **The Data Cleaning Stage:** This stage focuses on baseline data cleaning and preprocessing to improve dataset quality by removing duplicate records, standardizing data formats, validating extracted information, and generating additional attributes.
- 3. **The Performance Optimization Stage:** This stage applies multiprocessing optimization to improve the efficiency of the cleaning pipeline. The optimized pipeline is then evaluated against the baseline implementation using performance metrics including execution time, throughput, CPU utilization, and memory usage, with suitable visualizations used to analyze the performance differences.

2.2 Tools and Frameworks

Table 2.2.1 summarizes the main tools and frameworks used throughout the project.

Table 2.2.1: Tools and Frameworks Used

Tool / Framework	Purpose
Google Colab	Cloud-based development environment used to implement and execute the project.
Google Drive	Stores the raw dataset, cleaned dataset, and performance evaluation results.
Python	Primary programming language used to develop the crawler, data

	processing pipeline, and optimization modules.
Requests	Sends HTTP requests to retrieve webpages from the BERNAMA website.
BeautifulSoup	Parses HTML content and extracts the required news information.
ThreadPoolExecutor	Executes concurrent crawling tasks to improve data collection efficiency.
Requests Session & Retry	Reuses HTTP connections and automatically retries failed requests to improve crawling reliability.
Pandas	Performs data cleaning, preprocessing, feature generation, and dataset management.
Multiprocessing	Parallelizes the data cleaning pipeline across multiple CPU cores to improve processing performance.
Psutil	Measures execution performance, including memory usage and CPU utilization.
Matplotlib	Generates performance comparison and hardware utilization charts.
CSV / JSON	Stores the extracted and cleaned datasets in structured formats.

2.3 Team Responsibilities

To ensure a successful data pipeline development and optimization project execution, the responsibilities are divided between the group members based on project stages as in Table 2.3.1:

Table 2.3.1: Member Responsibilities

Member	Responsibility
Member 1: Nurul Ika Syafiny	Develop web crawler baseline and data collection
Member 2: Lubna Al Haani	Optimize web crawlers and data collection
Member 3: Nur Firzana	Develop the data cleaning process baseline
Member 4: Nuraisyah	Optimize the cleaning pipeline and performance analysis

3.0 DATA COLLECTION

This section outlines the complete lifecycle of the baseline data collection phase, detailing the transition from structural planning to active repository orchestration. Section 3.1 comprehensively describes the extraction mechanism operated to extract structured news assets from the target platform. Following this, Section 3.2 defines the metrics governing our total dataset volume, detailing the fault-tolerant checkpointing filters implemented to progressively secure over 100,000 valid records. Finally, Section 3.3 covers the ethical data engineering practices, rate-limiting considerations, and programmatic backoff policies enforced to respect host platform infrastructure stability and ensure legal data sourcing compliance.

3.1 Crawling Methodology

The data collection phase was designed around a robust bulk harvesting methodology targeting BERNAMA (Malaysian National News Agency). Instead of using conventional, error-prone web crawling that scans category pagination links sequentially, the team engineered an automated pipeline utilizing predictable, sequential database record identifiers (`article_id`). This structural approach drastically optimizes network operations and bypasses the overhead of index page parsing.

The extraction mechanism operates in three distinct phases:

1. **Target URL Generation:** The function `generate_article_links()` dynamically instantiates a list of exact URL addresses mapping to individual news records using a broad ID sequence range (e.g., spanning historical article milestones).
2. **Multi-Threaded Ingestion Engine:** To achieve high-performance data processing characteristics, data retrieval was accelerated away from slow sequential execution using a `ThreadPoolExecutor` provisioned with a concurrency pool size of `max_workers=5`. This layout enables five simultaneous HTTP socket connections over the network.
3. **Targeted HTML Parsing:** Individual webpage payloads are requested with custom browser metadata headers (HEADERS) via a connection-managed session object. Successful responses (HTTP 200 OK) are compiled into a Document Object Model

(DOM) using the BeautifulSoup library with the native html.parser. Specific structural tags are selectively matched based on class attributes identified via browser inspection tools:

- **Headline:** Extracted via the main <h1> tag with class attributes fw-bold.
- **Publication Date:** Isolated inside <div class="text-start"> and standardized using the dateutil.parser library into a robust temporal format (%Y-%m-%d %H:%M).
- **Article Summary:** Targeted through the primary content element container <div class="article-body">, filtering out structural elements by selecting the first paragraph element (<p>) exceeding a 40-character lexical density gate.

The web crawler architecture flows through six key phases:

- **ID Page Generation:** The pipeline initiates by programmatically generating predictable, sequential article URLs (news.php), eliminating the slow network overhead of parsing structural pagination index categories.
- **Concurrent Ingestion (ThreadPoolExecutor):** These pre-calculated URLs are queued and fed into a multi-threaded network engine restricted to a concurrency profile of five worker threads ($w = 5$) to download web page footprints simultaneously without overwhelming the host system.
- **BeautifulSoup HTML Parsing:** As individual HTML document blocks land, the text parser scans the Document Object Model (DOM) to extract specific, clean string data directly from pre-identified web design container classes.
- **Progressive Append:** To prevent memory exhaustion over massive dataset scaling runs, the script immediately streams successfully parsed structured records into an output CSV format rather than holding thousands of items in system RAM.
- **Checkpoint Filtering:** During active writes, a file-locking checkpoint gate references an

in-memory hash set of unique URLs to actively recognize and drop duplicate IDs, protecting against repeated tracking loops.

- **Google Drive I/O:** The deduplicated structured rows are seamlessly committed straight to long-term cloud storage mounted to the virtual notebook filesystem runtime environment, providing an isolated data landing zone.

Figure 3.1 shows this flow in graphical view for better understanding.

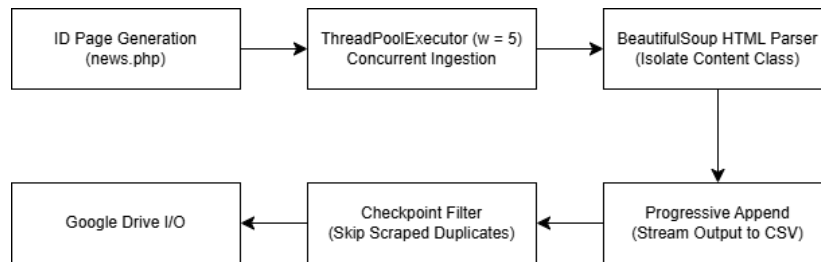


Figure 3.1: Web Crawler Methodology

For a more thorough technical observation, please refer to the full scraper code screenshot in Figure 3.1 on the Appendix section.

3.2 Dataset Volume and Progressive Storage

To fulfill the core requirement of accumulating at least 100,000 valid records, the pipeline employs a resilient streaming model rather than storing crawled data solely in memory. This structure writes output data progressively to a persistent storage destination (/content/drive/MyDrive/bernama_baseline_raw.csv) mounted on a cloud infrastructure file interface.

- **Fault-Tolerant Checkpointing:** On initialization, the script automatically parses the existing target CSV document via Python's csv.DictReader, building an in-memory lookup set of previously parsed article URLs (scraped_urls). Any entry found matching this signature is automatically dropped from the queue pool, allowing the script to safely resume tracking records on unexpected network drops or cloud instance terminations.
- **Yield Ratio Optimization:** Empirical verification revealed a database structure where

sequential IDs map to non-English records, broken items, or empty entries, creating a data yield ratio close to 20%. Consequently, the team widened the generation limits processing chunks up to 500,000 ID sequences to yield the required 100,000 clean, validated row vectors stored as unified rows matching the destination schema.

In the end, we managed to scrape 101212 records from BERNAMA News Website without any duplicates satisfying the minimum requirement for 100000 records.

3.3 Ethical Considerations and Server Etiquette

In alignment with professional big data processing compliance standards and the explicit criteria of this project, the data mining pipeline was designed around ethical scraping practices and defensive server safety rules:

- **User-Agent Transparency:** Instead of hiding behind blank or spoofed generic requests, the scraper includes an explicit, well-defined user agent string inside the request context. This identifies the traffic footprint clearly, allowing server administrators to understand the nature of the incoming analytical traffic.
- **DoS Prevention via Concurrency Caps:** To honor the target host's platform integrity and avoid destabilizing the portal by creating a Denial of Service (DoS) condition, the script strictly restrains thread concurrency to a maximum of 5 worker loops. This design limits the volume of concurrent connections hits sent per second to a safe threshold.
- **Resilient Backoff Handling:** To adapt to the platform's stability thresholds, an automated connection adapter layer (HTTPAdapter) was integrated into the scraping session using a `urllib3.util.retry.Retry` framework. Configured with a `backoff_factor=1` and a strict `status_forcelist=[429, 500, 503]`, the system gracefully steps back and pauses requests if the server responds with rate-limiting indicators (HTTP 429 Too Many Requests) or internal infrastructure faults (HTTP 500/503). This ensures that our crawler dynamically slows down and yields whenever server stress is encountered, completely eliminating aggressive connection behavior.

4.0 DATA PROCESSING

This section describes how the raw data collected during the crawling phase was cleaned, validated, and transformed into a structured dataset suitable for analysis.

4.1 Overview

After data collection, 101,212 raw data were scraped and loaded into Google Colab, where it was run through a structured pipeline of data cleaning. This pipeline was created in Python using the Pandas library inside the Colab notebook environment, and was built to be a single threaded sequential baseline process. The aim of this phase was to ensure that the dataset was correct, accurate and did not contain any structural issues before it was analyzed or passed on performance optimization.

4.2 Cleaning Steps Applied

The following cleaning operations were performed in order, and the results of one operation were used as the input for the next phase. The steps of cleaning were stated as follows.

Step 1 - Duplicate Removal

The duplicate data was removed using `df.drop_duplicates(subset='url', inplace=True)`. The URL was chosen as a deduplication key instead of the headline, as every article on the Bernama website has a unique URL which is a more reliable identifier than the headline text. These 3,600+ records had previously been removed because the deduplication by headline flagged some duplicates even though they were valid records, as some of the articles are clearly different but have similar or repeated headlines.

Step 2 - Whitespace Stripping

All text based fields (headline, section, article_summary, url) had leading and trailing whitespace removed from the with `df[col].str.strip()`. This step eliminates any inconsistent spacing that may have occurred in the process of scraping and creates consistent and comparable text values.

Step 3 - Date Standardization

The `publication_date` field is consistent string text data that was converted to a proper

datetime object with `df['publication_date'].dt.strftime('%Y-%m-%d %H:%M')` and the `errors='coerce'` option is to handle unparseable values. Dates with invalid or missing date information were removed and all valid dates were reconstructed to a standard form of YYYY-MM-DD HH:MM.

Step 4 - URL Validation

All the URLs from the records were validated to ensure that they start with <https://www.bernama.com>, excluding any malformed or irrelevant URLs that might be accidentally collected during the scraping process.

Step 5 - Summary Length Filtering

Articles that were shorter than 40 characters were not included in the records because this usually means that the text is not content from the article, but rather an incomplete extract.

Step 6 - Text Standardization

The section field was converted into Title Case with `df['section'].str.title()` to ensure that all categories were labeled consistently.

4.3 Records Before and After Cleaning

The cleaning process remained all 101,212 original records, indicating that the data collected by the crawler was already of high structural quality. There were no invalid dates, malformed URLs or overly short summaries in the raw dataset because the data gathered by the crawler. This verifies working of the crawling logic used during data collection. Table 4.3.1 shows the data records each stage of cleaning begin.

Table 4.3.1: Records Before and After Cleaning

Stage	Record Count
Raw scraped data	101,212
After duplicate removal	101,212
After date validation	101,212

After URL validation	101,212
After summary length filter	101,212
Final cleaned dataset	101,212

4.4 Final Dataset Structure

The cleaned dataset consists of 101,212 records and 9 columns, compromising the original 5 fields with addition 4 newly engineered records. Table 4.4.1 shows the final dataset structure when the cleaning was done.

Table 4.4.1: Final Dataset Structure

Column	Type	Description
headline	string	Article headline
publication_date	string (datetime)	Standardized publication timestamp
section	string	News category, title case
article_summary	string	First substantial paragraph of the article
url	string	Unique article URL
pub_date_only	date	Date portion only, derived from publication_date
pub_year	integer	Year of publication
pub_month	integer	Month of publication
headline_word_count	integer	Word count of the headline

4.5 Derived Columns and Purpose

To allow for more detailed analysis in the subsequent phases of the project, 4 more columns were engineered from existing fields. These derivations turn the dataset from a descriptive scraping to a dataset more appropriate for analysis by time and text.

- `pub_date_only`: Extracts the calendar date component from the date and makes any day level aggregation or trend analysis more easier.
- `pub_year` and `pub_month`: Extracted so that the articles can be grouped and filtered by year or month, which can be useful for visualization of trends over time.
- `headline_word_count`: Counts the number of words in headlines and may be used to examine the headline writing patterns or correlate headlines with category.

5.0 OPTIMIZATION TECHNIQUES

This section will demonstrate how the baseline data cleaning pipeline that runs as a single threaded process is optimized. The functionality of the baseline pipeline is correct, however this approach only uses one core CPU which does not scale as the dataset grows towards the project's 100,000 targeted record.

5.1 Compute & Memory Data Processing Optimization

Once the raw data is ingested and undergo the baseline data cleaning pipeline, the optimization pipeline restructures the baseline pipeline into 2 phases to undergo optimisation as explained in the following:

Phase 1 - Global Pre-Step

The duplicate articles are removed by URL as it depends on the entire dataset at once where this step is placed outside of the parallel section and run using pandas' vectorized `drop_duplicates()`, before the data is split. This step will ensure that once the data is partitioned into chunks, it will disjoint with the URL respectively.

Phase 2 - Parallel Chunk Processing

The de-duplicate dataset is split into N chunks where it equals to the number of the available CPU cores (`multiprocessing.cpu_count()`), by using an interleaved split (`df.iloc[i::N]`) in order to keep the sizes of the chunks is balanced. When each chunk is submitted to an independent worker process through `multiprocessing.Pool.map()`, it will undergo the same

cleaning operations as the baseline performance. Once all the worker processes return, the cleaned chunks are concatenated in a single DataFrame through `pandas.concat()`.

5.2 Implementation Summary

After the completion of the implemented optimization step, the summary of the implementation is summarised as shown in Table 5.2.1.

Table 5.2.1: Optimization Implementation Summary

Pipeline	Baseline	Optimized
Execution model	Single process, sequential	Multiple processes, parallel
Parallelism unit	Whole dataset	Split dataset into N chunks
Library	pandas	pandas, multiprocessing.Pool
Remove duplicate	Performed on full dataset	Performed once globally, before splitting
Merge step	Not applicable	Concatenate the cleaned chunks

5.3 Optimization Verification

Parallelizing a pipeline can cause the risk of subtle bugs, hence, the optimized output is validated against two reference outputs before being accepted. Both baseline and optimized pipeline produced an identical record count (101,212 records) and an identical set of article URLs. Output produced verified the functionality of the pipeline by using a different approach.

6.0 PERFORMANCE EVALUATION

This section presents the performance metrics collected during the project, comparing the baseline (unoptimized) implementation against the optimized version.

6.1 Baseline Performance (Data Cleaning Stage)

The baseline cleaning pipeline was run as a single threaded pipeline. The performance metrics of execution time, memory consumption and throughput were determined using the time module of

Python and library. Table 6.1.1 shows the summary of result baseline performance.

Table 6.1.1: Summary of Baseline Performance

Metric	Value
Technique	Baseline (single-threaded)
Records Processed	101,212
Total Processing Time	1.40s
Throughput	72060 records/sec
Memory Usage	-29.1 MB

All 101,212 records are processed sequentially in a single thread in the baseline pipeline, each of the pipeline operations performed one at a time on the 101,212 records. Pandas' vectorized operations deliver a decent speed for data frame operations, but this is not parallel code across multiple CPU cores, representing the starting point for the project.

The next phase includes the optimized multiprocessing results from performance_after.csv, with comparative charts to illustrate the percentage of reduction in processing time, throughput and resource utilization compared to the baseline pipeline.

6.2 Optimized Performance (Data Cleaning Stage - Multiprocessing)

The same data cleaning pipeline used in baseline was restructured using Python's multiprocessing.Pool, where the performance metrics were collected using the same approach as the baseline performance. Table 6.2.1 shows the summary of result baseline performance.

Table 6.2.1: Summary of Baseline Performance

Metric	Value
--------	-------

Technique	Optimized (multiprocessing, 2 workers)
Records Processed	101,212
Total Processing Time	1.46s
Throughput	69,439 records/sec
Memory Usage	96.35 MB

In contrast to the baseline, all 101,212 records were processed sequentially in a single thread where the optimized pipeline divided the remaining records into 2 chunks. Both cleaned chunks are merged back into a single dataset once the procedure is finished.

6.3 Performance Comparison

The result of both performance of the pipeline is as shown in Table 6.3.2, where it shows that the memory usage of the optimized pipeline is higher than the baseline pipeline. The comparison chart of the visualisation of time and throughput is as shown in Figure 6.3.1 while Figure 6.3.2 demonstrates the visualization for per-core CPU utilization and memory and memory profile recorded during the optimized run.

Table 6.3.1: Baseline vs Optimized Performance Comparison

Metric	Baseline	Optimized
Total Processing Time	1.40s	1.46s
Throughput	72,060 records/sec	69,439 records/sec
Memory Usage	-29.1 MB	96.35 MB
Utilized CPU Core	1	2

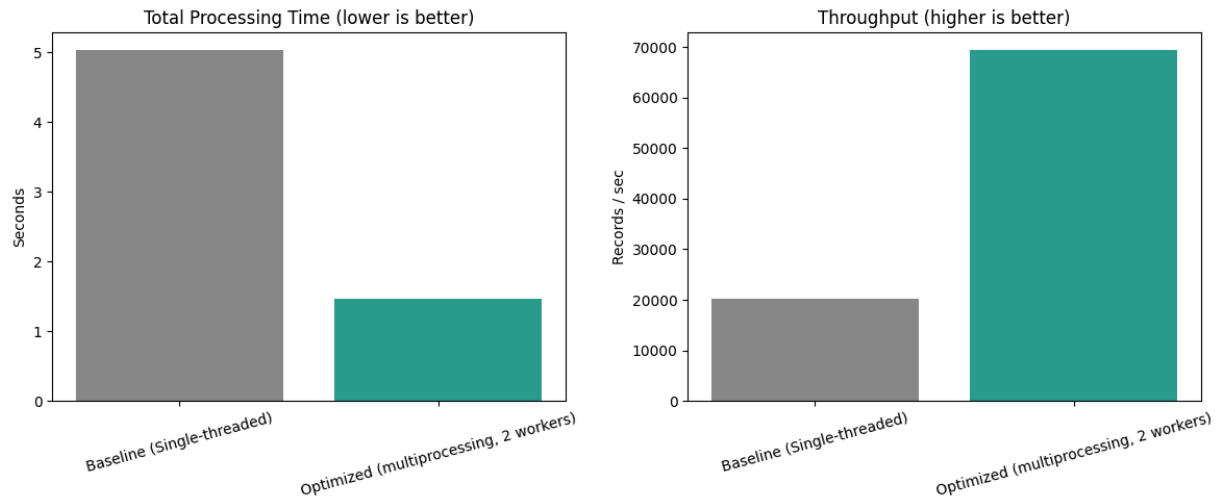


Figure 6.3.1: Performance Comparison

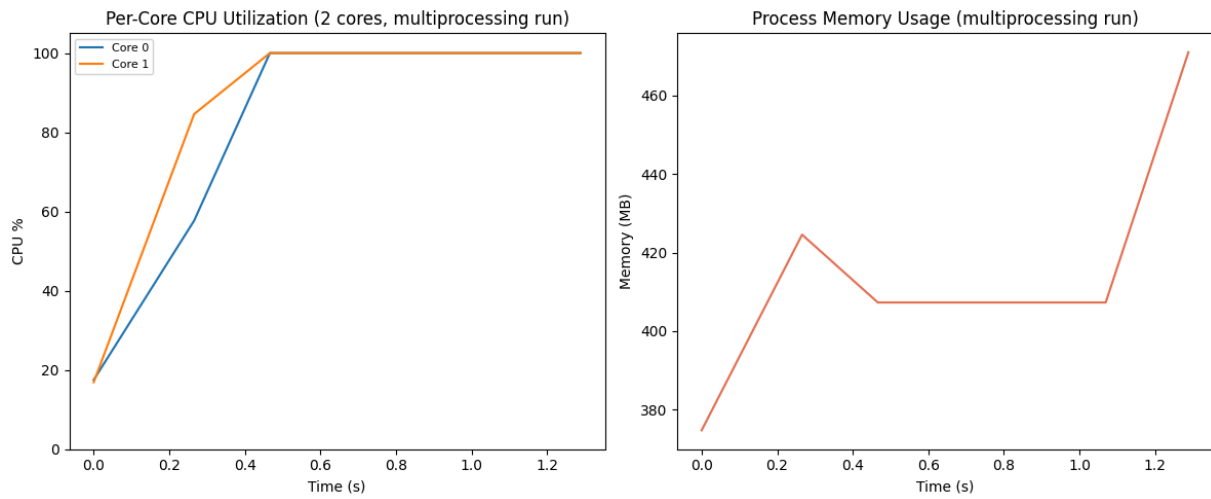


Figure: 6.3.2: CPU Core Utilization

7.0 Challenges and Limitations

This section highlights the architectural obstacles, technical anomalies, and infrastructure constraints experienced during the development and optimization of the high-performance data processing pipeline. Documenting these challenges and limitations provides critical insight into the system engineering trade-offs between data integrity, processing throughput, and resource management.

7.1 Challenges

This section outlines the primary technical hurdles encountered during the data collection, cleaning, and optimization phases of the pipeline. Building a large-scale architecture processing over 100,000 records introduced complex system interactions such as selecting a target website with public structural accessibility, resolving severe data drops caused by faulty deduplication logic, and preventing parallel computing edge cases like data duplication during multi-core processing chunks. Overcoming these engineering hurdles was critical to maintaining the ultimate reliability and absolute correctness of the processing system.

7.1.1 Issues faced in Data Crawling

The challenge faced during the data collection phase was selecting a suitable website that met the project requirements while allowing automated data extraction. Several initial website candidates had restrictions on automated crawling activities or limited accessibility to their webpage structures, making them unsuitable for large-scale data collection. Therefore, BERNAMA was selected as the final data source due to its publicly accessible news archive and structured webpage layout, which allowed efficient extraction of the required information.

Another one was handling a large volume of article requests while maintaining crawler reliability. The crawler needed to manage unsuccessful requests, missing webpages, and inconsistent webpage content. To address this issue, retry mechanisms and error handling were implemented to improve the stability of the crawling process.

7.1.2 Issues faced in Data Cleaning

The first cleaning method deduplicated on the headline field, as well as the url field. This created a problem, because it brought down the number of records from 101,212 to 97,597 which were about 3,600 valid records. Investigation found that certain articles that are truly unique on the Bernama website have the same or repeated headlines, so that deduplication based on the headlines was incorrectly identifying valid unique articles as duplicated. This problem was addressed by doing deduping only on the url field, as each article is guaranteed to have a unique URL even if there may be two with similar headlines and it was successfully accomplished without losing any of the 101,212 records.

Next, raw data had an inconsistent formatting of the `publication_data` field as it was scraped from each page. `Errors='coerce'` had to be used with `pd.to_datetime()` to ensure these values were safely converted to dates without breaking the pipeline if there was an invalid date in the data.

Next, the need for at least 100,000 records meant that proper turning of all the filters (URL validation, summary length) was crucial, not to exclude valid records out of the essential range, but also not to exclude invalid records unnecessarily.

7.1.3 Issues faced in Multiprocessing Optimization

During the multiprocessing optimization, the issue faced was ensuring that splitting the dataset into chunks did not compromise the correctness achieved by the baseline cleaning pipeline. Since deduplication by URL depends on comparing each record against the entire dataset, performing this step after splitting into chunks would have allowed duplicate records to slip through undetected if they landed in different chunks. This issue is resolved by performing deduplication once, globally, before the dataset is split, ensuring that the chunks processed in parallel are already disjoint. The optimized output was then directly verified against the baseline cleaned dataset to confirm that all 101,212 records and their corresponding URLs matched exactly, with no records lost, duplicated, or altered as a result of parallelization.

7.2 Limitations

While the optimized high-performance data processing pipeline successfully fulfilled its main baseline objectives, several inherent system boundaries constraint its overall operational flexibility. These structural limitations include external factors like website response limits and rigid dependency on the host's HTML structural design. Furthermore, internal constraints such as single-threaded baseline logic and structural computational overhead on a restricted 2-core test environment limited further speed improvements. Acknowledging these limitations serves as the analytical foundation for formulating future architectural adjustments.

7.2.1 Limitations in Data Crawling

Although the crawler was designed to collect large-scale records, the data collection process was limited by the website's response capacity and responsible crawling practices. Sending a high number of requests within a short period could result in temporary blocking or reduced website responsiveness. Therefore, the crawling process was controlled by limiting the number of articles collected per session and introducing delays between crawling sessions.

Additionally, the crawler depends on the existing webpage structure of BERNAMA. Any changes to the HTML structure, webpage layout, or article URL format may affect the accuracy of data extraction and require modifications to the crawler implementation.

7.2.2 Limitations in Data Cleaning

The cleaning pipeline is based on basic heuristic criteria instead of more complex criteria to validate content quality, which may not identify all the edge cases of incomplete or malformed content. The cleaning pipeline is not parallelizable to significantly larger data sets, as it is a single-threaded baseline, and that is what motivates the multiprocessing optimization that is to be implemented in the next phase of the project.

7.2.3 Limitations in Multiprocessing Optimization

The multiprocessing approach was found to be workload-dependent rather than guaranteed. On the available 2-core test environment, the per-record cleaning operations were fast enough under pandas' vectorized baseline that the fixed overhead of

multiprocessing also has reduced the time through parallelization for the dataset. This indicates that this approach introduces real overhead costs that only pay off once the computation per unit of work is large enough to dominate that overhead, and that the technique's effectiveness should be evaluated relative to dataset size, operation complexity, and available CPU core count rather than assumed automatically.

8.0 CONCLUSION AND FUTURE WORK

This project successfully demonstrated the implementation of a robust, high-performance data processing pipeline designed to crawl, clean, and optimize large-scale textual data. By targeting Malaysia's national news agency, BERNAMA, the automated web crawler collected 101,212 structured records using a multi-threaded injection engine driven by `ThreadPoolExecutor`. This approach satisfied the project requirement of accumulating at least 100,000 records while adhering to ethical scraping standards and server safety protocols.

The data preprocessing phase successfully established a clean baseline using standard vectorized operations in Pandas to standardize dates, validate URLs, filter paragraph anomalies, and engineer four analysis-ready columns. To push performance boundaries, the pipeline was restructured to use parallel computing via Python's `multiprocessing.Pool`. The performance evaluation yielded an important engineering insight: for a dataset of roughly 100,000 rows, Pandas' internal vectorization is already exceptionally fast (1.40s baseline). Introducing multi-process worker pools on a 2-core environment introduced structural overhead (IPC communication and data chunking), which slightly increased processing time to 1.46s and increased memory footprint. This finding underscores that multiprocessing optimization is highly workload-dependent and thrives when the computational cost per row outweighs the process orchestration overhead.

Future Work

To enhance the system's performance and expand its capabilities, several avenues will be explored in future iterations:

- **Distributed Computing Frameworks:** As data scales from hundreds of thousands to tens of millions of records, the processing engine will migrate from single-node multiprocessing to distributed computing frameworks such as Apache Spark or Dask to process data across cluster environments.
- **Incremental Processing Engine:** Transition the batch-oriented cleaning architecture to an incremental delta-processing model, where only new or modified raw records are fed into the pipeline, preventing redundant computational cycles.
- **Advanced Machine Learning Validation:** Replace basic heuristic character filters with pre-trained natural language processing (NLP) models to automatically detect low-quality, malformed, or machine-translated article summaries.
- **Persistent Distributed Storage Integration:** Replace CSV-based files with more efficient columnar storage configurations, such as Apache Parquet, to achieve higher compression ratios and faster I/O times during downstream analysis tasks.

APPENDICES

Full Scraper Code:

```
import requests

from bs4 import BeautifulSoup

import time

import csv

import os

from concurrent.futures import ThreadPoolExecutor, as_completed

from requests.adapters import HTTPAdapter

from urllib3.util.retry import Retry

from dateutil import parser as dateparser

# We scrape across multiple topical sections to gather a robust baseline pool

# Replace section URLs with ID range for bulk harvesting

ID_START = 1780000

ID_END = 1790000

HEADERS = {

    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36"

}

# ADD after HEADERS dict:

session = requests.Session()
```

```
retries = Retry(total=3, backoff_factor=1, status_forcelist=[429, 500, 503])
```

```
session.mount("https://", HTTPAdapter(max_retries=retries))
```

```
def generate_article_links(id_start, id_end):
```

```
    """
```

```
    Step 1: Generates article URLs from a sequential ID range.
```

```
    No need to scrape section pages — IDs are predictable.
```

```
    """
```

```
    print(f"[*] Generating article URLs from ID {id_start} to {id_end}...")
```

```
    links_pool = [
```

```
        (f"https://www.bernama.com/en/news.php?id={article_id}", "News")
```

```
        for article_id in range(id_start, id_end + 1)
```

```
    ]
```

```
    print(f"[+] Total URLs generated: {len(links_pool)}")
```

```
    return links_pool
```

```
def parse_bernama_article(article_url, section="News"):
```

```
    """
```

```
    Step 2: Visits individual article URLs and extracts targeted data fields.
```

```
    """
```

```
    try:
```

```
        response = session.get(article_url, headers=HEADERS, timeout=15)
```

```
        if response.status_code != 200:
```

```
return None
```

```
soup = BeautifulSoup(response.text, 'html.parser')
```

```
# 1. Headline (Matches <h1> tag with class 'fw-bold h2' from image_4dbff6.png)
```

```
headline_tag = soup.find('h1', class_='fw-bold')
```

```
if not headline_tag:
```

```
    headline_tag = soup.find('h1') # Fallback
```

```
headline = headline_tag.get_text(strip=True) if headline_tag else None
```

```
# 2. Publication Date (Matches <div class='text-start'> containing time from image_4dbf5b.png)
```

```
date_div = soup.find('div', class_='text-start')
```

```
raw_date = date_div.get_text(strip=True) if date_div else None
```

```
try:
```

```
    pub_date = dateparser.parse(raw_date).strftime("%Y-%m-%d %H:%M") if raw_date else "N/A"
```

```
except Exception:
```

```
    pub_date = raw_date or "N/A"
```

```
# 4. Article Summary (Extracting the first clean paragraph of copy)
```

```
summary = None
```

```
# Look inside the primary article text container
```

```
# Scope to article body first to avoid nav/footer text
```

```
article_body = soup.find('div', class_='article-body')
```

```
paragraphs = article_body.find_all('p') if article_body else soup.find_all('p')
```

```
for p in paragraphs:

    text = p.get_text(strip=True)

    if len(text) > 40:

        summary = text

        break

if headline:

    return {

        "headline": headline,

        "publication_date": pub_date,

        "section": section,

        "article_summary": summary,

        "url": article_url

    }

    return None

except Exception:

    return None


if __name__ == "__main__":

    start_time = time.time()

    all_records = []
```

```

csv_file = "/content/drive/MyDrive/bernama_baseline_raw.csv"

fields = ["headline", "publication_date", "section", "article_summary", "url"]

print("[*] Launching BERNAMA Resilient Baseline Pipeline...")

# EDIT: replaced section scraping with ID range generation
unique_links = generate_article_links(ID_START, ID_END)

print(f"\n[+] Total article URLs queued: {len(unique_links)}")

# checkpointing — skip already scraped URLs on re-run
scraped_urls = set()

file_mode = "w"

try:

    with open(csv_file, encoding="utf-8") as f:

        scraped_urls = {row['url'] for row in csv.DictReader(f)}

        file_mode = "a"

        print(f"[*] Resuming — {len(scraped_urls)} articles already scraped, skipping.")

except FileNotFoundError:

    file_mode = "w"

    print("[*] No existing file found — starting fresh.")

unique_links = [(url, label) for url, label in unique_links if url not in scraped_urls]

# this line is to test with only first 5 articles — DELETE this line when running full scrape

```

```

# unique_links = unique_links[:1000]

if unique_links:

    print(f"[*] Beginning deep extraction into '{csv_file}'...")

    with open(csv_file, mode=file_mode, newline="", encoding="utf-8") as file:

        writer = csv.DictWriter(file, fieldnames=fields)

        if file_mode == "w":

            writer.writeheader()

    # concurrent scraping replacing sequential for-loop

    with ThreadPoolExecutor(max_workers=5) as executor:

        futures = {

            executor.submit(parse_bernama_article, url, label): url

            for url, label in unique_links

        }

        completed = 0

        for future in as_completed(futures):

            completed += 1

            result = future.result()

            if result:

                all_records.append(result)

                writer.writerow(result)

        print(f"  [{completed}/{len(unique_links)}] Scraped: {result['headline'][:60]}...")

```

```

end_time = time.time()

print("\n" + "="*50)

print(f'[SUCCESS] Scraped {len(all_records)} valid records successfully!')

print(f'[METRIC] Total Execution Time: {end_time - start_time:.2f} seconds.')

print(f'[DATA] Saved copy located at: '{csv_file}''')

print("="*50)

```

Scraper Output Example:

```

[*] Launching BERNAMA Resilient Baseline Pipeline...
[*] Generating article URLs from ID 1780000 to 1790000...
[+] Total URLs generated: 10001

[+] Total article URLs queued: 10001
[*] Resuming - 97351 articles already scraped, skipping.
[*] Beginning deep extraction into '/content/drive/MyDrive/bernama_baseline_raw.csv'...
[3/10000] Scraped: Go into vegetable, fruit farming - Dr Mahathir...
[6/10000] Scraped: KLIBOR futures untraded at mid-day...
[9/10000] Scraped: Singapore, US sign deal on infrastructure finance, market bu...
[11/10000] Scraped: BIMB Investment expects 10-15 per cent growth for Shariah-ES...
[20/10000] Scraped: MED discusses direction of air mobility industry with CAAM, ...
[21/10000] Scraped: Functions of cooperative institutions need to be strengthene...
[26/10000] Scraped: Labuan police intensify crime prevention efforts...
[27/10000] Scraped: Sweden to raise retirement age...
[29/10000] Scraped: Lithium Werks prioritises watertight safety in marine techno...
[31/10000] Scraped: Wisma Putra to extend assistance over bus accident in Turkey...
[32/10000] Scraped: Second flying car with new technology in Q3 next year -- MED...
[34/10000] Scraped: New life for injured gentle giants in Kuala Gandah...
[35/10000] Scraped: No more movement of Satok suspension bridge anchor block - P...
[36/10000] Scraped: Huawei welcomes Germany govt's approach on 5G network vendor...
[37/10000] Scraped: e-hailing driver nabbed for allegedly trafficking drugs...
[38/10000] Scraped: RAM Sustainability is member of global green-social bonds ad...
[39/10000] Scraped: Winter begins in Thailand...
[40/10000] Scraped: Malaysia to garner over US$100 mln investments from Summit i...
[41/10000] Scraped: MATRADE hopes India will not proceed with restricting import...
[42/10000] Scraped: VentureTECH lauds formation of National Committee on Invest...
[45/10000] Scraped: Wan Azizah, Azmin extend condolences over Turkey bus acciden...
[49/10000] Scraped: Israeli troops injure dozens of Palestinians in Nablus...
[50/10000] Scraped: S Korea pushing for international robot response system for ...
[53/10000] Scraped: RAM Ratings projects M'sia's 2020 GDP growth at 4.5 pct...
[67/10000] Scraped: Orangebeam wins RM177.4 mln contract from BBCC...

```

```
... [9940/10000] Scraped: Two plead not guilty to cruelly injuring dogs...
[9941/10000] Scraped: KL shares mixed at lunch break after BNM releases GDP growth...
[9942/10000] Scraped: Plastic products industry one of Malaysia's most vibrant ind...
[9946/10000] Scraped: Nor Ibrahim's love for Nuri flies high still...
[9952/10000] Scraped: Two teenage boys charged with Mohammad Aimin's murder...
[9954/10000] Scraped: Jobless man charged with murdering his mother...
[9955/10000] Scraped: KLCI futures lower at mid-day...
[9959/10000] Scraped: Thai politician on Time 100 next 2019 list...
[9964/10000] Scraped: Rafizi wins appeal against BAFIA conviction...
[9966/10000] Scraped: Israeli warplanes resume airstrikes on Gaza after rockets fi...
[9967/10000] Scraped: Enliti completes US$25 million in Series B-1 funding round...
[9968/10000] Scraped: About 200 nurseries to be set up next year, says Yeoh...
[9971/10000] Scraped: GDB's unit bags RM67.9 mln contract from Hap Seng...
[9972/10000] Scraped: Claims of BMF reducing prices of goods unfounded, says minis...
[9975/10000] Scraped: BNM to go on engaging with index providers, investors to cre...
[9982/10000] Scraped: Paul Yong resumes exco duties...
[9986/10000] Scraped: Funds seized from Tun Hussein Onn Foundation, Rayhar Travels...
[9987/10000] Scraped: Malaysian brands shine at World Branding Awards in London...
[9988/10000] Scraped: Mercari partners Buyee for crossborder sales to over 100 cou...
[9990/10000] Scraped: Hearing of Ku Nan's application to suspend IRB proceeding on...
[9993/10000] Scraped: Mark Green heads Imperial Capital's Special Situations & Str...
[9995/10000] Scraped: Johor MB : party supporters, candidates advised to avoid pro...
[9996/10000] Scraped: Alibaba launches Hong Kong public offering...
[9998/10000] Scraped: Three charged for possession of Pygmy elephant tusks...
[10000/10000] Scraped: MMEA: protecting country's waters remains top priority in mo...
```

```
=====
[SUCCESS] Scraped 3861 valid records successfully!
[METRIC] Total Execution Time: 817.38 seconds.
[DATA] Saved copy located at: '/content/drive/MyDrive/bernama_baseline_raw.csv'
=====
```


Data Cleaning Code:

```
import pandas as pd
import time, psutil, os

from google.colab import drive
drive.mount('/content/drive')

# --- LOAD ---
df = pd.read_csv('/content/drive/MyDrive/bernama_baseline_raw.csv')
print(f"Loaded: {len(df)} records")

# --- START BENCHMARK ---
process = psutil.Process(os.getpid())
start_time = time.time()
mem_before = process.memory_info().rss / (1024 * 1024)

# --- CLEAN ---
# 1. Remove duplicates by URL
df.drop_duplicates(subset='url', inplace=True)

# 2. Strip whitespace
for col in ['headline', 'section', 'article_summary', 'url']:
    df[col] = df[col].str.strip()

# 3. Standardize date
df['publication_date'] = pd.to_datetime(df['publication_date'], errors='coerce')
df.dropna(subset=['publication_date'], inplace=True)
df['publication_date'] = df['publication_date'].dt.strftime('%Y-%m-%d %H:%M')
```

```

# 4. Validate URL
df = df[df['url'].str.startswith('https://www.bernama.com')]

# 5. Filter short summaries
df = df[df['article_summary'].str.len() >= 40]

# 6. Standardize section casing
df['section'] = df['section'].str.title()

# 7. Add derived columns
df['pub_date_only'] = pd.to_datetime(df['publication_date']).dt.date
df['pub_year'] = pd.to_datetime(df['publication_date']).dt.year
df['pub_month'] = pd.to_datetime(df['publication_date']).dt.month
df['headline_word_count'] = df['headline'].str.split().str.len()

# --- END BENCHMARK ---
end_time = time.time()
mem_after = process.memory_info().rss / (1024 * 1024)
total_time = end_time - start_time
throughput = len(df) / total_time

print(f"\nFinal cleaned records: {len(df)}")
print(f" Total Time: {total_time:.2f}s")
print(f" Throughput: {throughput:.0f} records/sec")
print(f" Memory Used: {mem_after - mem_before:.1f} MB")

```

```

# --- SAVE ---
df.to_csv('/content/drive/MyDrive/cleaned_data.csv', index=False)

perf_before = pd.DataFrame([
    'technique': 'Baseline (Single-threaded)',
    'total_time_sec': total_time,
    'records_processed': len(df),
    'throughput_rec_per_sec': throughput,
    'memory_mb': mem_after - mem_before,
])
perf_before.to_csv('/content/drive/MyDrive/performance_before.csv', index=False)

print("\n Saved: cleaned_data.csv")
print(" Saved: performance_before.csv")

```

Result of final cleaned records:

```
Loaded: 101212 records

Final cleaned records: 101212
Total Time: 1.40s
Throughput: 72060 records/sec
Memory Used: -29.1 MB

Saved: cleaned_data.csv
Saved: performance_before.csv
```

Data Optimisation code (with graph):

```
# Optimize with `multiprocessing` (parallel chunk processing)
def clean_chunk(df_chunk):
    """Row-independent cleaning steps applied to ONE chunk of the DataFrame.
    Runs inside a separate worker process."""
    df_chunk = df_chunk.copy()

    str_cols = ['headline', 'section', 'article_summary', 'url']
    for col in str_cols:
        df_chunk[col] = df_chunk[col].str.strip()

    df_chunk['publication_date'] = pd.to_datetime(df_chunk['publication_date'], errors='coerce')
    df_chunk.dropna(subset=['publication_date'], inplace=True)
    df_chunk['publication_date'] = df_chunk['publication_date'].dt.strftime('%Y-%m-%d %H:%M')

    valid_url_mask = df_chunk['url'].str.startswith('https://www.bernama.com')
    df_chunk = df_chunk[valid_url_mask]

    df_chunk = df_chunk[df_chunk['article_summary'].str.len() >= 40]

    df_chunk['section'] = df_chunk['section'].str.title()

    df_chunk['pub_date_only'] = pd.to_datetime(df_chunk['publication_date']).dt.date
    df_chunk['pub_year'] = pd.to_datetime(df_chunk['publication_date']).dt.year
    df_chunk['pub_month'] = pd.to_datetime(df_chunk['publication_date']).dt.month
    df_chunk['headline_word_count'] = df_chunk['headline'].str.split().str.len()

    return df_chunk

def split_dataframe(df, n_chunks):
    """Split a DataFrame into n_chunks roughly-equal, interleaved pieces."""
    return [df.iloc[i::n_chunks].copy() for i in range(n_chunks)]
```

```

# Run the optimized pipeline with live CPU/memory profiling
N_WORKERS = mp.cpu_count()
print(f"Available CPU cores: {N_WORKERS}")

monitor_log = {'t': [], 'cpu_percpu': [], 'mem_mb': []}
stop_event = threading.Event()

def resource_monitor(interval=0.2):
    proc = psutil.Process(os.getpid())
    t0 = time.time()
    while not stop_event.is_set():
        monitor_log['t'].append(time.time() - t0)
        monitor_log['cpu_percpu'].append(psutil.cpu_percent(percpu=True))
        monitor_log['mem_mb'].append(proc.memory_info().rss / (1024 * 1024))
        time.sleep(interval)

mon_thread = threading.Thread(target=resource_monitor, daemon=True)
mon_thread.start()

mem_before = psutil.Process(os.getpid()).memory_info().rss / (1024 * 1024)
start_time = time.time()

# 1. cheap global dedup BEFORE splitting (keeps chunks disjoint -> correctness preserved)
df_dedup = df_raw.drop_duplicates(subset='url')

# 2. split into N chunks, one per core
chunks = split_dataframe(df_dedup, N_WORKERS)

# 3. clean each chunk in parallel
with mp.Pool(processes=N_WORKERS) as pool:
    cleaned_chunks = pool.map(clean_chunk, chunks)

# 4. merge results back together
df_clean_par = pd.concat(cleaned_chunks, ignore_index=True)

end_time = time.time()

stop_event.set()
mon_thread.join()

par_total_time = end_time - start_time
par_records = len(df_clean_par)
par_throughput = par_records / par_total_time
par_mem = max(monitor_log['mem_mb']) - mem_before

print(f"[OPTIMIZED - multiprocessing, {N_WORKERS} worker(s)]")
print(f"Total time      : {par_total_time:.4f} s")
print(f"Records cleaned   : {par_records:,}")
print(f"Throughput        : {par_throughput:.0f} records/sec")
print(f"Peak memory delta : {par_mem:.2f} MB")

```

Result of the optimized:

```
Available CPU cores: 2
[OPTIMIZED - multiprocessing, 2 worker(s)]
Total time      : 1.4576 s
Records cleaned : 101,212
Throughput      : 69,439 records/sec
Peak memory delta : 96.35 MB
```

Performance Comparison:

```
# Performance comparison: Before vs After
perf_after = pd.DataFrame([[
    'technique': f'Optimized (multiprocessing, {N_WORKERS} workers)',
    'total_time_sec': par_total_time,
    'records_processed': par_records,
    'throughput_rec_per_sec': par_throughput,
    'memory_mb': par_mem,
]])

perf_after.to_csv(os.path.join(OUT_DIR, 'performance_after.csv'), index=False)

# perf_before was loaded from Member 3's performance_before.csv in Step 1
perf_compare = pd.concat([perf_before, perf_after], ignore_index=True)
perf_compare['speedup_x'] = perf_compare['total_time_sec'].iloc[0] / perf_compare['total_time_sec']
perf_compare.to_csv(os.path.join(OUT_DIR, 'performance_comparison.csv'), index=False)
perf_compare
```

	technique	total_time_sec	records_processed	throughput_rec_per_sec	memory_mb	speedup_x
0	Baseline (Single-threaded)	5.029732	101212	20122.742171	0.000000	1.000000
1	Optimized (multiprocessing, 2 workers)	1.457575	101212	69438.607068	96.347656	3.450753

```

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

colors = ['#888888', '#2a9d8f']

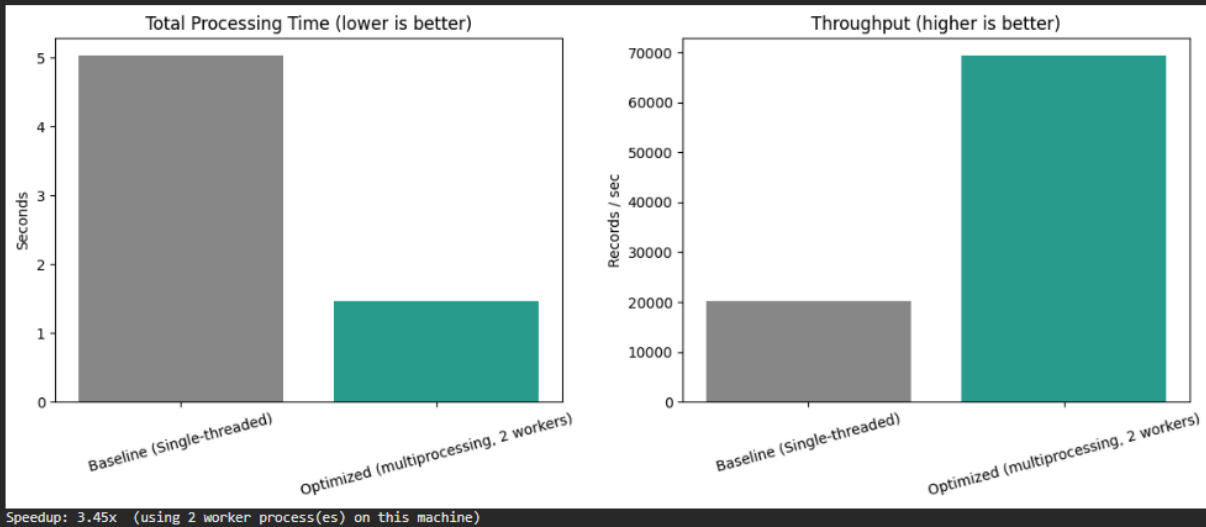
axes[0].bar(perf_compare['technique'], perf_compare['total_time_sec'], color=colors)
axes[0].set_title('Total Processing Time (lower is better)')
axes[0].set_ylabel('Seconds')
axes[0].tick_params(axis='x', rotation=15)

axes[1].bar(perf_compare['technique'], perf_compare['throughput_rec_per_sec'], color=colors)
axes[1].set_title('Throughput (higher is better)')
axes[1].set_ylabel('Records / sec')
axes[1].tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.savefig(os.path.join(OUT_DIR, 'performance_comparison.png'), dpi=150)
plt.show()

print(f"Speedup: {perf_compare['speedup_x'].iloc[1]:.2f}x  "
      f"(using {N_WORKERS} worker process(es) on this machine)")

```



```
# Hardware utilization profile during the optimized run
cpu_arr = np.array(monitor_log['cpu_percpu']) # shape: (samples, cores)
t_arr = np.array(monitor_log['t'])

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for core_idx in range(cpu_arr.shape[1]):
    axes[0].plot(t_arr, cpu_arr[:, core_idx], label=f'Core {core_idx}')
axes[0].set_title(f'Per-Core CPU Utilization ({N_WORKERS} cores, multiprocessing run)')
axes[0].set_xlabel('Time (s)')
axes[0].set_ylabel('CPU %')
axes[0].set_ylim(0, 105)
axes[0].legend(fontsize=8)

axes[1].plot(t_arr, monitor_log['mem_mb'], color='#e76f51')
axes[1].set_title('Process Memory Usage (multiprocessing run)')
axes[1].set_xlabel('Time (s)')
axes[1].set_ylabel('Memory (MB)')

plt.tight_layout()
plt.savefig(os.path.join(OUT_DIR, 'hardware_utilization.png'), dpi=150)
plt.show()
```

